

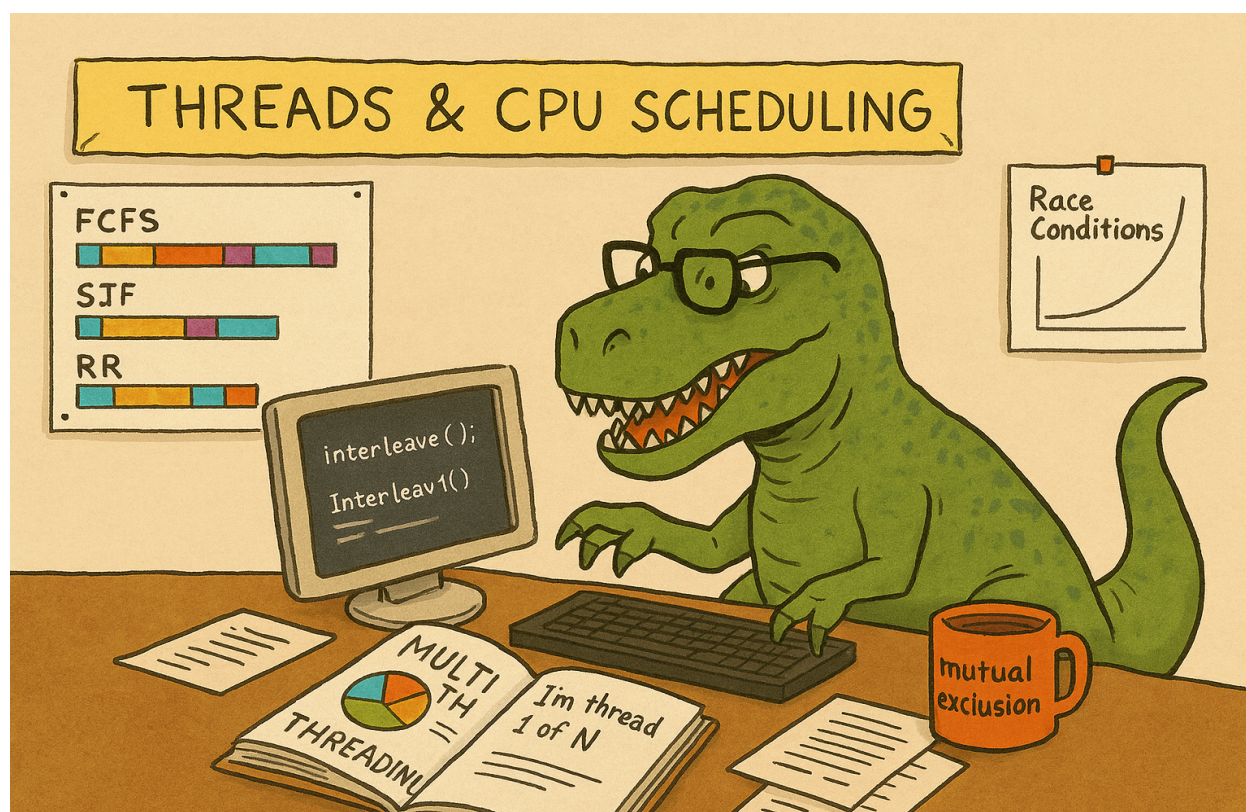


سیستم عامل - بهار ۱۴۰۴

استاد: دکتر مهدی کارگهی

پروژه سوم: چندریسه‌ای، فرآیندها، رفتار همزمان، زمان‌بندی

طراحان: محمد امانلو، کوثر شیرینی جعفرزاده، علیرضا حسینی



اهداف

در این تمرین مفاهیم فرآیند، ترد، رفتار همزمان و زمان‌بندی CPU را تجربه می‌کنیم. از یک برنامه‌ی ساده با چند ترد شروع می‌کنیم و می‌بینیم که چگونه سیستم‌عامل خروجی را به شکل غیرقابل‌پیش‌بینی interleave می‌کند، بعد روی یک متغیر مشترک یک race condition واقعی می‌سازیم و آن را برطرف می‌کنیم، و در ادامه با پیاده‌سازی یک شبیه‌ساز کوچک زمان‌بندی، سه الگوریتم کلاسیک FCFS، SJF و Round Robin را روی چند سناریوی از پیش‌تعریف‌شده اجرا و با هم مقایسه می‌کنیم.

بخش اول: از اولین Thread تا ساخت و رفع Race Condition

در این سؤال، با تردها آشنا می‌شوید، interleaving را در عمل می‌بینید، روی یک مثال ساده‌ی عددی چندنخی کار می‌کنید و یک race condition واقعی می‌سازید و با mutex حلش می‌کنید.

۱-الف) آشنایی با Thread و interleaving

برنامه‌ای بنویسید که:

1. از کاربر عدد N (تعداد تردها) را بگیرد.
2. N ترد بسازد.
3. هر ترد قبل از چاپ پیام، یک sleep تصادفی کوتاه (۰ تا ۳ ثانیه) انجام دهد و سپس پیام I'm thread i of N را چاپ کند. که در آن i شماره‌ی ترد است (مثلاً از ۰ تا $N-1$ یا ۱ تا N).
4. در انتها، ترد اصلی با pthread_join منتظر اتمام همه‌ی تردها بماند.
5. توضیح دهید چرا ترتیب چاپ پیام‌ها با شماره‌ی تردها یکی نیست و چرا نمی‌توانید ترتیب دقیق را پیش‌بینی کنید. به مفاهیمی مثل زمان‌بندی سیستم‌عامل، interleaving و «این‌که هر ترد ممکن است در هر نقطه‌ای preempt شود» اشاره کنید.

۱-ب) جمع آرایه‌ی بزرگ – مقایسه‌ی تک‌تردی و چندتردی

برنامه‌ای بنویسید که:

1. یک آرایه‌ی بزرگ از اعداد صحیح تصادفی تولید کند (مثلاً $N = 10^7$).
2. نسخه‌ی تک‌تردی (Single-thread):
 - a. با یک حلقه‌ی ساده، جمع کل آرایه را حساب کند؛
 - b. زمان اجرا را اندازه بگیرد (مثلاً با clock_gettime یا هر روش دیگر).
3. نسخه‌ی چندتردی (Multi-thread):
 - a. تعداد تردها را T (مثلاً ۲، ۴، ۸) در نظر بگیرید؛
 - b. آرایه را به T بخش مساوی (یا تقریباً مساوی) تقسیم کنید؛
 - c. برای هر بخش یک ترد بسازید تا جمع بخش خودش را حساب کند؛
 - d. ترد اصلی مجموع جمع‌های جزئی را جمع بزند؛
 - e. زمان اجرای این نسخه را هم اندازه بگیرید.

4. توضیح دهید آیا همیشه نسخه‌ی چندتردی سریع‌تر است یا نه؛ اگر نه، حدس بزنید علت چیست؟
5. برای چند اندازه‌ی مختلف آرایه از کم تا بسیار زیاد و سه مقدار مختلف از T زمان نسخه‌ی تک‌تردی و چندتردی را کنار هم قرار دهید و از روی جدول، تقریباً مشخص کنید از چه اندازه‌ی N به بعد، نسخه‌ی چندتردی شروع می‌کند بهتر از نسخه‌ی تک‌تردی عمل کند. این مرز را به صورت حدودی در گزارش توضیح دهید و بگویید چرا برای کارهای خیلی کوچک، تک‌تردی به صرفه‌تر است و برای کارهای بزرگ، چندتردی.
6. مفهوم دانه‌بندی (granularity) چیست؟ چرا اگر کار هر ترد خیلی ریز باشد، سربار ساخت و مدیریت تردها خودش از سود موازی‌سازی بیشتر می‌شود.
7. (اختیاری) اگر امکانش را دارید، همین آزمایش را روی هر دو سیستم عامل Windows و Linux انجام دهید.
8. توضیح دهید چه عواملی می‌تواند باعث تفاوت زمان و مرز ریزدانی در تقسیم وظایف روی تردها در دو سیستم عامل مختلف شود. چرا در بخش 7 مرزهای دانه‌بندی در دو سیستم عامل متفاوت است؟

۱-ج) چند ترد و یک کانتر مشترک – ساختن Race Condition

برنامه‌ای بنویسید که:

1. یک متغیر global با مقدار اولیه صفر داشته باشد.
2. دو پارامتر M (تعداد تردها) و K (تعداد افزایش‌ها) را یا از ورودی بگیرد، یا به صورت ثابت تنظیم کند.
3. M ترد بسازد.
4. هر ترد در یک حلقه‌ی ساده و بدون هیچ قفل یا همگام‌سازی، دقیقاً K بار مقدار متغیر global را افزایش می‌دهد.
5. پس از پایان همه‌ی تردها (join)، مقدار counter را چاپ کند و با مقدار مورد انتظار $M * K$ مقایسه کند.
6. برنامه را چند بار برای یک M و K ثابت (مثلاً $M = 4, K = 100000$) اجرا کنید و خروجی‌ها را یادداشت کنید.
7. توضیح دهید چرا این وضعیت یک race condition است؟
8. یک global pthread_mutex_t تعریف و مقداردهی اولیه کنید.

9. با استفاده از mutex تعریف شده مشکل race condition را حل کنید. و برنامه را برای مقادیر بزرگتر مثل $M = 20$, $K = 1000000$ نیز اجرا کنید.
10. توضیح دهید critical section در این برنامه کجاست و چرا قرار دادن lock/unlock اطراف آن، مشکل را حل می‌کند.

بخش دوم: شبیه‌ساز زمان‌بندی CPU: FCFS, SJF, Round Robin

در این سؤال، یک شبیه‌ساز ساده‌ی زمان‌بندی CPU می‌نویسید که روی چند لیست پروسس از پیش تعریف‌شده، سه الگوریتم FCFS, SJF, RR را اجرا و مقایسه کنید.

همه‌چیز در user-space با آرایه و حلقه شبیه‌سازی می‌شود؛ قرار نیست واقعاً فرآیندهای سیستم را عوض کنید. برای سادگی می‌توانید تعداد پروسس‌ها و فهرست آن‌ها را در خود برنامه (hard-code) کنید (نیازی به ورودی از کاربر نیست).

برنامه‌ای بنویسید که:

1. برای هر پروسس سه مقدار داشته باشد:
 - pid (نام/شماره‌ی پروسس، مثلاً P1, P2, ...)
 - arrival_time
 - burst_time
 2. دو الگوریتم را پیاده‌سازی کند:
 - FCFS (غیر پیش‌دستانه)
 - i. پروسس‌ها بر اساس زمان ورود (arrival_time) در صف آماده قرار می‌گیرند؛
 - ii. هر بار که CPU آزاد می‌شود، اولین پروسس صف را تا پایان burst اجرا می‌کند.
 - SJF غیر پیش‌دستانه
 - i. وقتی CPU آزاد است، از میان پروسس‌های «رسیده و تمام‌نشده»، پروسسی را انتخاب می‌کند که کمترین burst_time را دارد؛
 - ii. پروسس انتخاب‌شده تا انتها اجرا می‌شود (بدون preemption).
 3. برای هر پراسس:
 - start_time و finish_time را حساب کنید؛
 - برای هر پراسس $waiting_time = start_time - arrival_time$ را محاسبه کنید.
 - برای هر پراسس $turnaround_time = finish_time - arrival_time$ را محاسبه کنید.
 - میانگین waiting_time و turnaround_time را چاپ کنید.
- حالا به همان برنامه، یک الگوریتم Round Robin پیش‌دستانه اضافه کنید که:
1. همان ورودی پروسس‌ها (arrival + burst) را استفاده کند؛

2. یک $\text{time quantum} = q$ از ورودی بگیرد یا در برنامه ثابت تعیین کنید (مثلاً $q = 1$ و $q = 4$ را تست کنید)؛

3. یک صف آماده (Ready Queue) نگه دارد و شبیه‌سازی کند که:

a. هر بار که نوبت یک پروسس می‌شود، حداکثر q واحد زمان اجرا می‌شود؛

b. در هر واحد زمان، remaining_time آن پروسس یک واحد کم می‌شود؛

c. اگر remaining_time صفر شد، پروسس تمام می‌شود و از سیستم خارج می‌شود؛

d. اگر هنوز تمام نشده باشد، به انتهای صف آماده برمی‌گردد؛

e. پروسس‌های تازه‌وارد ($\text{arrival_time} == \text{current_time}$) در طول شبیه‌سازی به صف اضافه می‌شوند.

4. میانگین waiting_time و turnaround_time را چاپ کنید.

برای ارزیابی برنامه پیاده سازی شده خود از سناریوهای زیر استفاده کنید:

سناریو اول:

Process	Arrival	Burst
P1	0	8
P2	0	4
P3	0	1
P4	0	3

سناریو دوم:

Process	Arrival	Burst
P1	0	7
P2	2	4
P3	4	1
P4	5	4

سناریو سوم:

Process	Arrival	Burst
P1	0	20
P2	0	3
P3	0	3

نتایج SJF، FCFS و RR (با حداقل دو مقدار q) را برای حداقل دو سناریوی بالا در یک جدول مقایسه کنید. از هر الگوریتم یک گانت چارت متنی بنویسید. توضیح دهید:

- اثرات q های کوچک و بزرگ را باهم مقایسه کنید.
- در سناریوهای شما، کدام الگوریتم از نظر تأخیر متوسط و عدالت بهتر عمل کرده است و چرا.

نکات و نحوهٔ تحویل

- کدهای شما می‌بایست به زبان C++/C بوده و با استفاده از Makefile و کامپایلر g++ قابل کامپایل و اجرا باشند.
 - پروژهٔ شما می‌بایست در سیستم عامل Linux و در زمان معقول قابل اجرا باشد.
 - تمامی کدها و فایل‌های خود را به صورت zip شده و به فرمت **OS-CA3-<SID>.zip** در صفحهٔ درس بارگذاری کنید.
 - در صورت داشتن سوال می‌توانید از طریق فروم یا گروه درس و یا شرکت در جلسات رفع اشکال سوالات خود را مطرح کنید.
 - تمامی مواردی که در فروم یا گروه درس و یا جلسات رفع اشکال به آن اشاره می‌شود جزئی از این تمرین خواهند بود.
 - این تمرین صرفاً برای یادگیری شما طراحی شده است. در صورت محرز شدن هرگونه تخلف، مطابق قوانین درس با آن برخورد خواهد شد.
- موفق باشید!